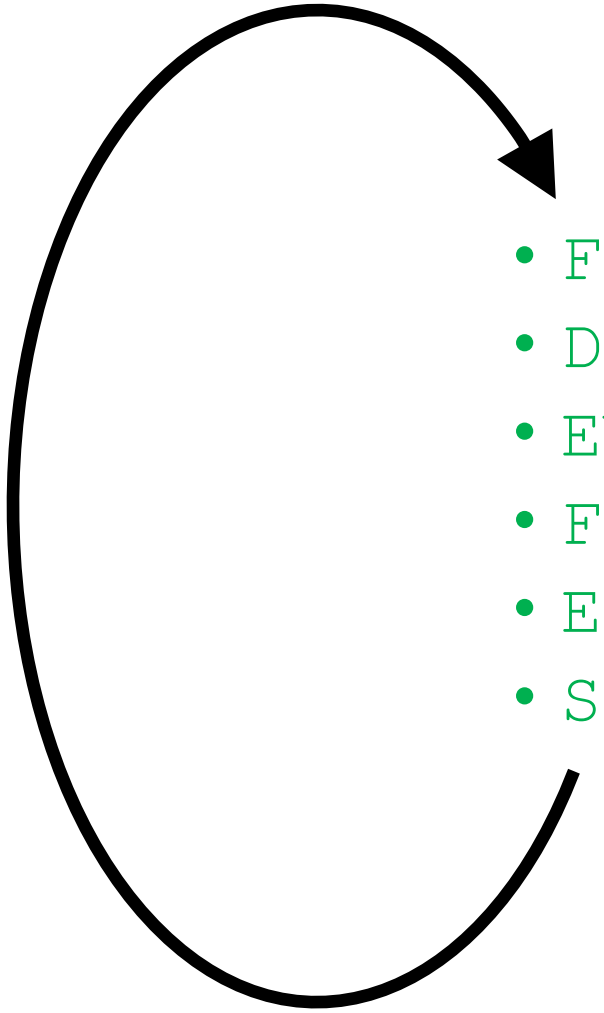
Assembly Programming

The Instruction Cycle



- FETCH
- DECODE
- EVALUATE ADDRESS
- FETCH OPERANDS
- EXECUTE
- STORE RESULT

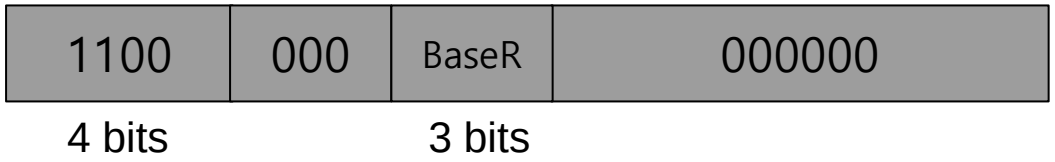
Changing the Sequence of Execution

- A computer program executes in sequence (i.e., in program order)
  - First instruction, second instruction, third instruction and so on
- Unless we change the sequence of execution
- Control instructions allow a program to execute out of sequence
  - They can change the PC by loading it during the EXECUTE phase
  - That wipes out the incremented PC (loaded during the FETCH phase)

Jump in LC-3

- Unconditional branch or jump
- LC-3

```
JMP   R2
```



- BaseR = Base register
- PC ← R2 (Register identified by BaseR)

This is register addressing mode

- Variations
  - RET: special case of JMP where BaseR = R7
  - JSR, JSRR: jump to subroutine

LC-3 Data Path

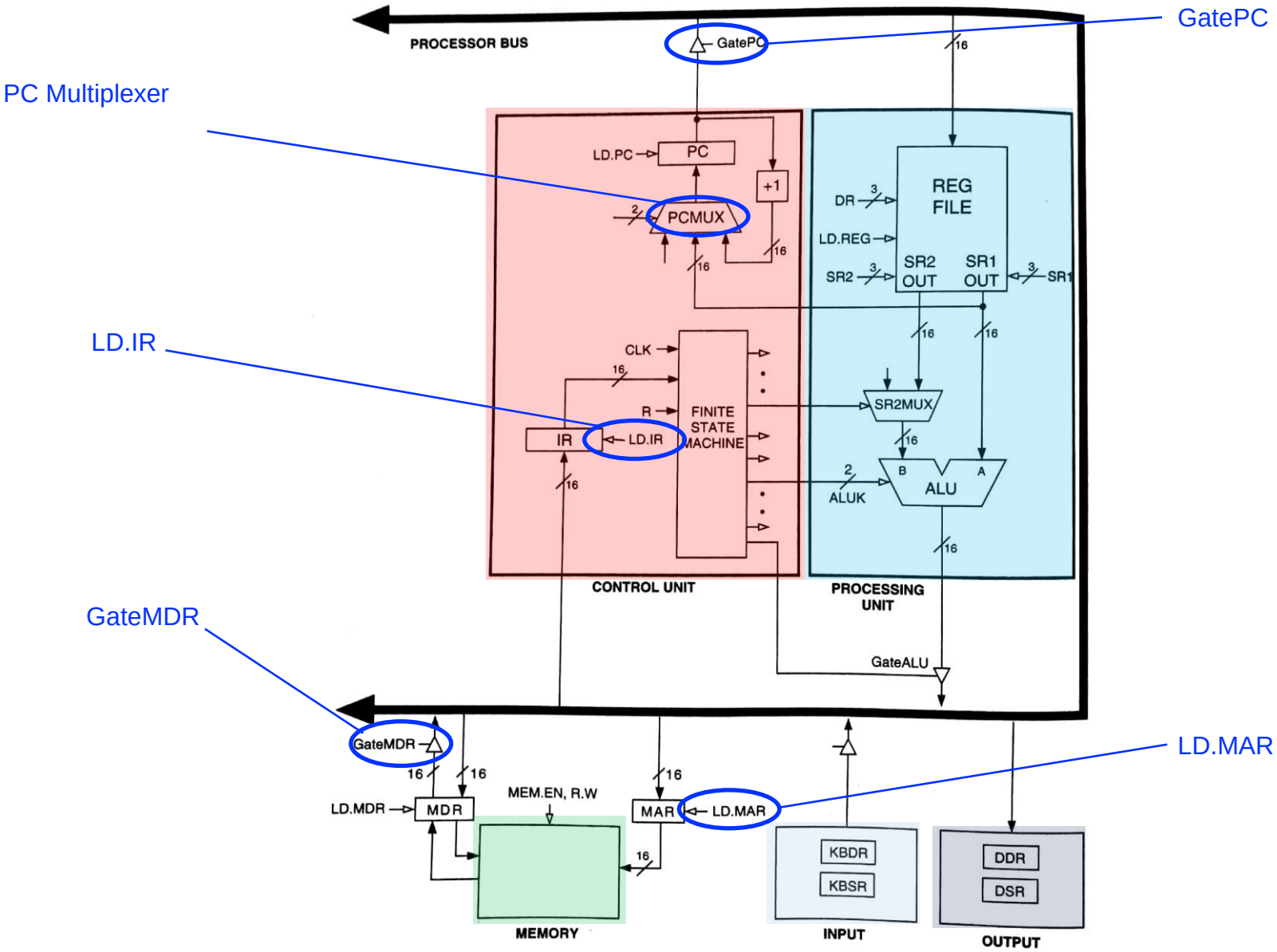


Figure 4.3 The LC-3 as an example of the von Neumann model

Control of the Instruction Cycle

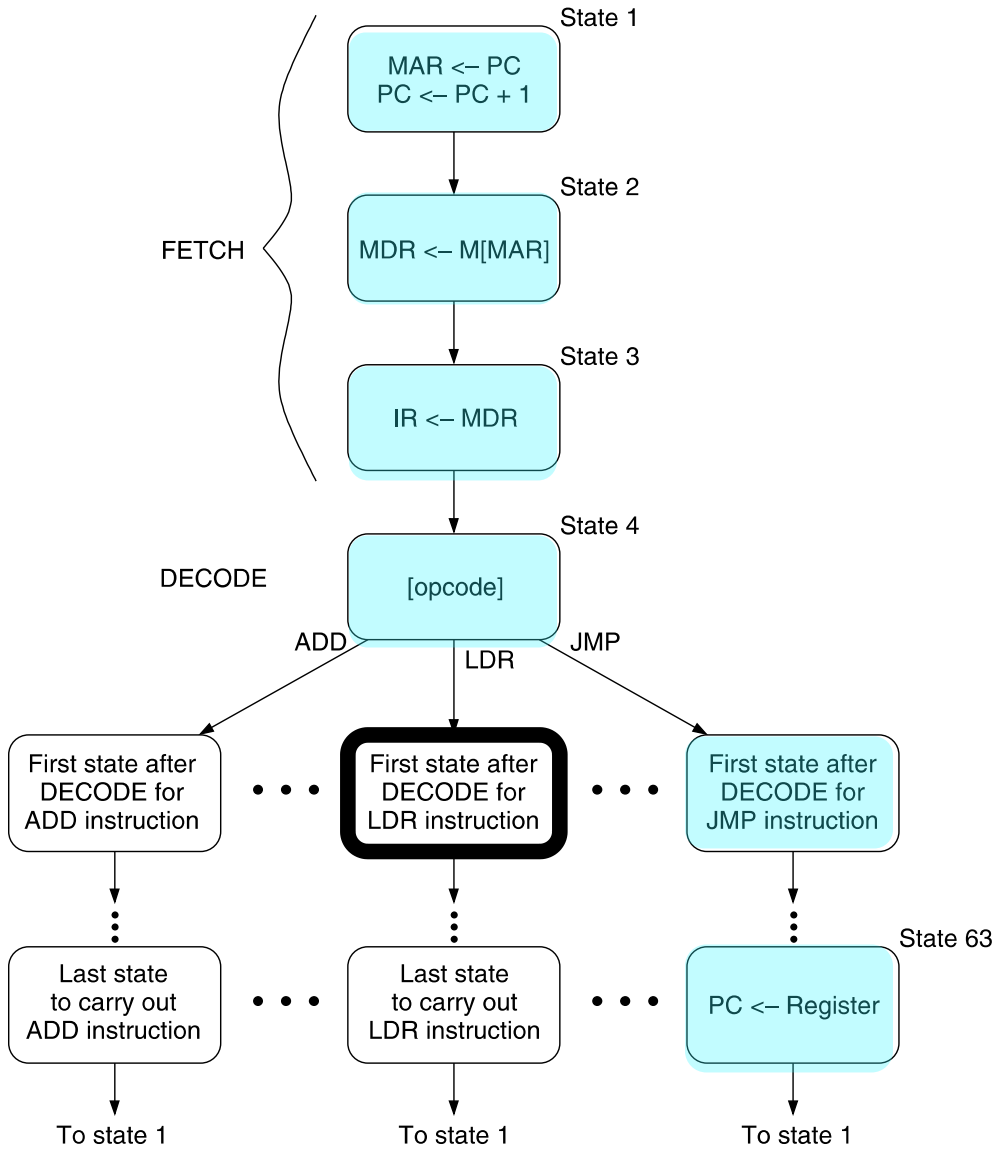


Figure 4.4 An abbreviated state diagram of the LC-3

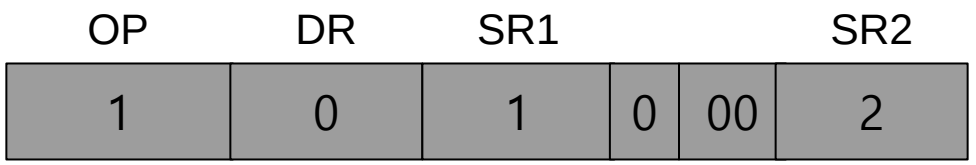
- State 1
  - The FSM asserts GatePC and LD.MAR
  - It selects input (+1) in PCMUX and asserts LD.PC
- State 2
  - MDR is loaded with the instruction
- State 3
  - The FSM asserts GateMDR and LD.IR
- State 4
  - The FSM goes to next state depending on opcode
- State 63
  - JMP loads register into PC

LC-3
Instruction Set Architectures

The Instruction Set

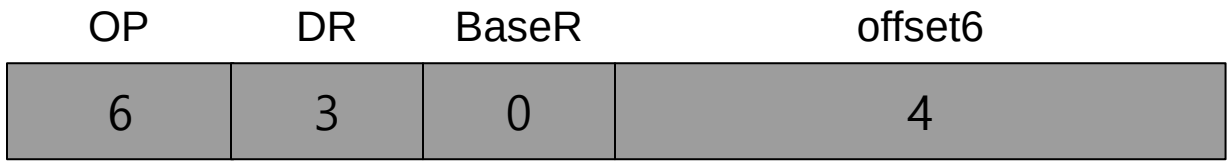
- It defines opcodes, data types, and addressing modes
- ADD and LDR have been our first examples

ADD



Register mode

LDR

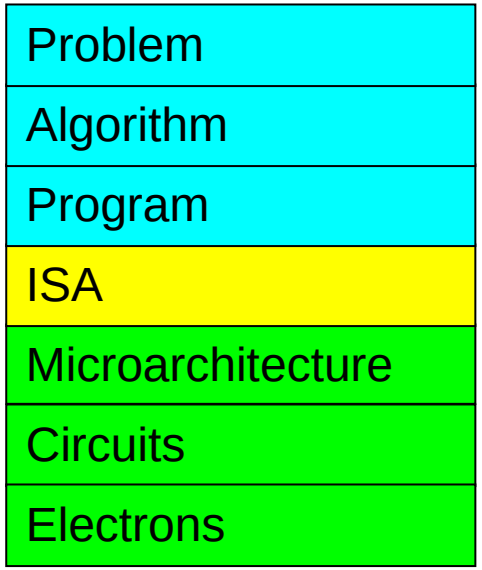


Base+offset mode

The Instruction Set Architecture

- The ISA is the interface between what the software commands and what the hardware carries out

- The ISA specifies
  - The memory organization
    - Address space (LC-3: 2^{16}, MIPS: 2^{32})
    - Addressability (LC-3: 16 bits, MIPS: 32 bits)
    - Word- or Byte-addressable
  - The register set
    - R0 to R7 in LC-3
    - 32 registers in MIPS
  - The instruction set
    - Opcodes
    - Data types
    - Addressing modes



Opcodes

- Large or small sets of opcodes could be defined
- Tradeoffs are involved
  - Hardware complexity vs. software complexity
- In LC-3 and in MIPS there are three types of opcodes
  - Operate
  - Data movement
  - Control

Opcodes in LC-3

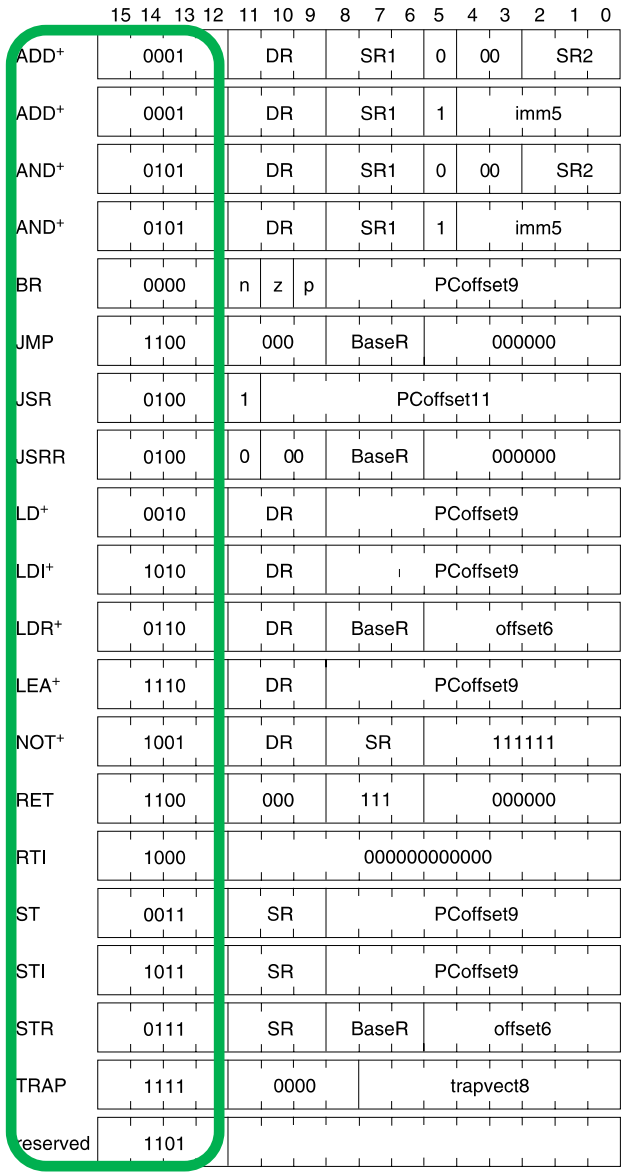


Figure 5.3 Formats of the entire LC-3 instruction set. NOTE: + indicates instructions that modify condition codes

Data Types

- An ISA supports one or several data types
- LC-3 only supports 2's complement integers
- MIPS supports
  - 2's complement integers
  - Unsigned integers
  - Floating point
- Again, tradeoffs are involved

Data Type Tradeoffs

- What is the benefit of having more or high-level data types in the ISA?
- What is the disadvantage?

- Think compiler/programmer vs. microarchitect

- Concept of semantic gap
  - Data types coupled tightly to the semantic level, or complexity of instructions

- Example: Early RISC architectures vs. Intel 432
  - Early RISC (e.g., MIPS): Only integer data type
  - Intel 432: Object data type

Addressing Modes

- An addressing mode is a mechanism for specifying where an operand is located

- There five addressing modes in LC-3
  - Immediate or literal (constant)
    - The operand is in some bits of the instruction
  - Register
    - The operand is in one of R0 to R7 registers
  - Three of them are memory addressing modes
    - PC-relative
    - Indirect
    - Base+offset

Operate Instructions

Readings

- *Introduction to Computing Systems: From Bits and Gates to C and Beyond by* Patt & Patel
  - Chapter 4, 5
  - Appendix A – LC3 ISA

Operate Instructions

- In LC-3, there are three operate instructions
  - NOT is a unary operation (one source operand)
    - It executes bitwise NOT
  - ADD and AND are binary operations (two source operands)
    - ADD is 2's complement addition
    - AND is bitwise SR1 & SR2

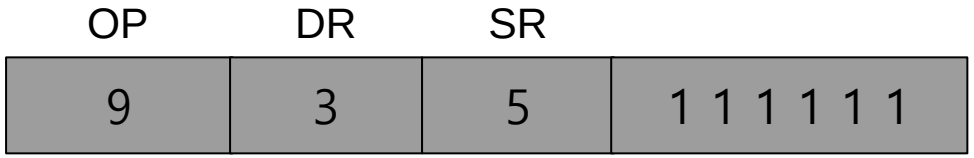
NOT in LC-3

- NOT assembly and machine code

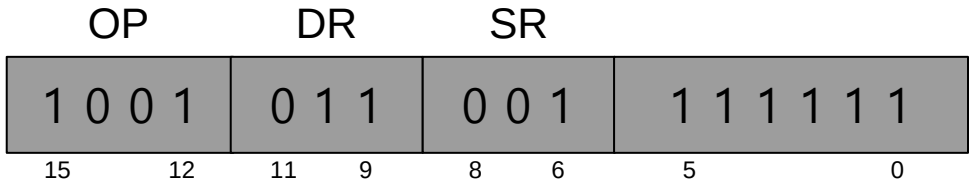
LC-3 assembly

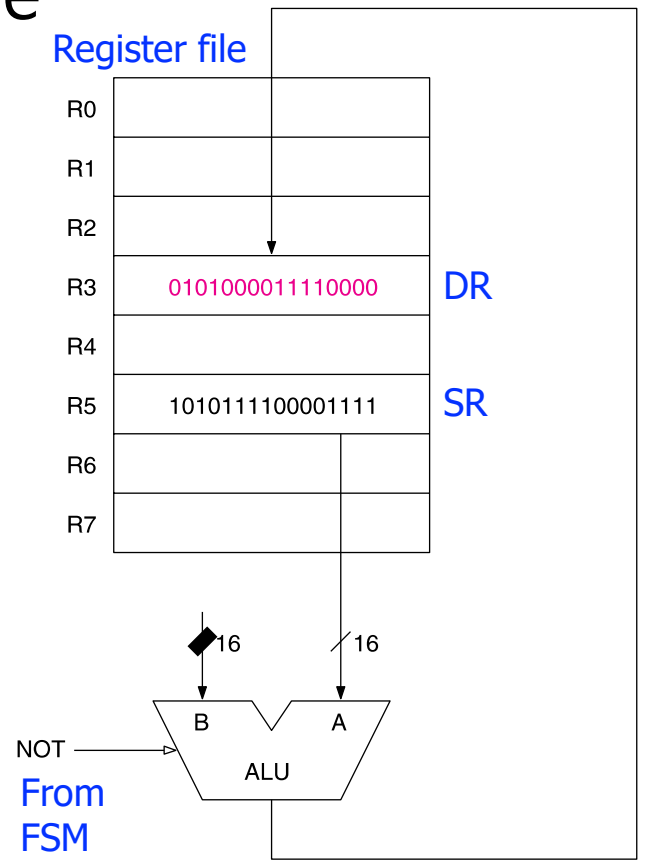
```
NOT   R3, R5
```

Field Values



Machine Code



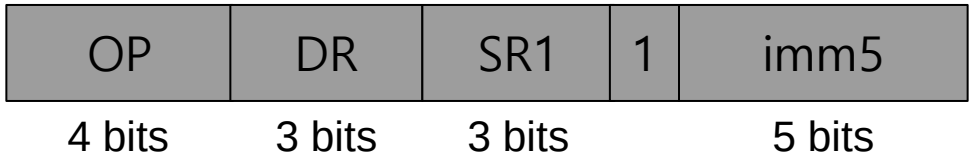


Operate Instructions

- We are already familiar with LC-3's ADD and AND with register mode (R-type in MIPS)
- Now let us see the versions with one literal (i.e., immediate) operand
- Subtraction is another necessary operation
  - How is it implemented in LC-3?

Operate Instr. with one Literal in LC-3

■ADD and AND



- ❑OP = operation
  - ■ E.g., ADD = 0001 (same OP as the register-mode ADD)
    - ❑ DR ← SR1 + sign-extend(imm5)
  - ■ E.g., AND = 0101 (same OP as the register-mode AND)
    - ❑ DR ← SR1 AND sign-extend(imm5)
- ❑SR1 = source register
- ❑DR = destination register
- ❑imm5 = Literal or immediate (sign-extend to 16 bits)

ADD with one Literal in LC-3

- ADD assembly and machine code

LC-3 assembly

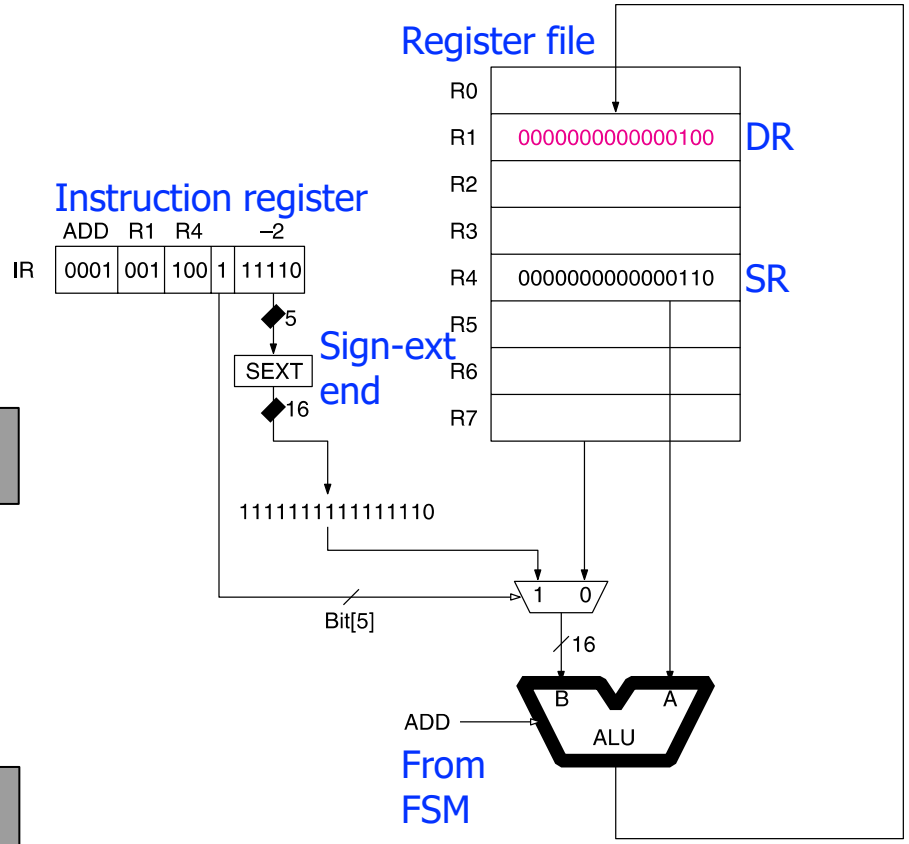
```
ADD R1, R4, #-2
```

Field Values

OP	DR	SR	imm5
1	1	4	1	-2

Machine Code

OP	DR	SR	imm5
0 0 0 1	0 0 1	1 0 0	1	1 1 1 1 0
15 12	11 9	8 6	5	4 0



Subtract in LC-3

- LC-3 assembly

High-level code

```
a = b + c - d;
```

LC-3 assembly

```
ADD   R2, R0, R1
NOT   R4, R3
ADD   R5, R4, #1
ADD   R6, R2, R5
```

2's complement of R4

- Tradeoff in LC-3
  - More instructions
  - But, simpler control logic

Subtract Immediate

- LC-3

High-level code

```
a = b - 3;
```

LC-3 assembly

```
ADD  R1, R0, #-3
```

Data Movement Instructions and Addressing Modes

Opcodes

- Large or small sets of opcodes could be defined
- Tradeoffs are involved
  - Hardware complexity vs. software complexity
- In LC-3 and in MIPS there are three types of opcodes
  - Operate
  - Data movement
  - Control

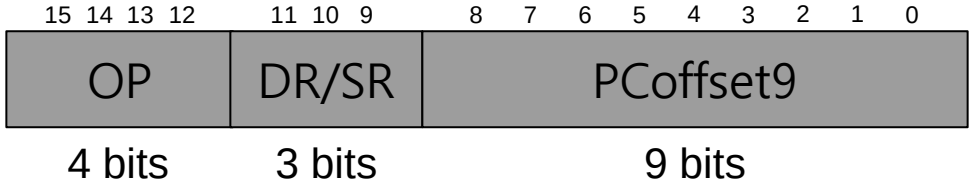
Data Movement Instructions

- In LC-3, there are seven data movement instructions
  - LD, LDR, LDI, LEA, ST, STR, STI

- Format of load and store instructions
  - Opcode (bits [15:12])
  - DR or SR (bits [11:9])
  - Address generation bits (bits [8:0])
  - Four ways to interpret bits, called addressing modes
    - PC-Relative Mode
    - Indirect Mode
    - Base+offset Mode
    - Immediate Mode

PC-Relative Addressing Mode

■LD (Load) and ST (Store)



❑OP = opcode
- ■ E.g., LD = 0010
- ■ E.g., ST = 0011

❑DR = destination register in LD
❑SR = source register in ST

❑LD: DR ← Memory[PC† + sign-extend(PCoffset9)]

❑ST: Memory[PC† + sign-extend(PCoffset9)] ← SR

† This is the incremented PC

LD in LC-3

- LD assembly and machine code

LC-3 assembly

```
LD R2, 0x1AF
```

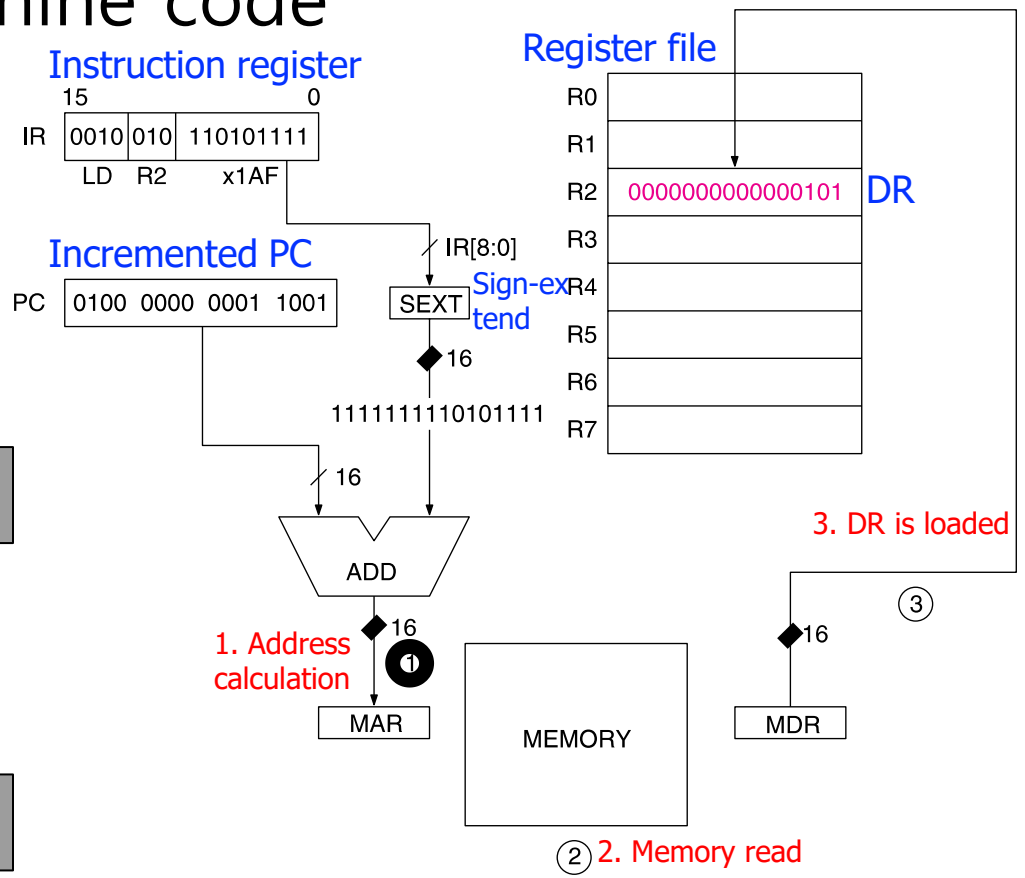
Field Values

OP	DR	PCoffset9
2	2	0x1AF

Machine Code

OP	DR	PCoffset9
0 0 1 0	0 1 0	1 1 0 1 0 1 1 1 1

15 12 11 9 8 0

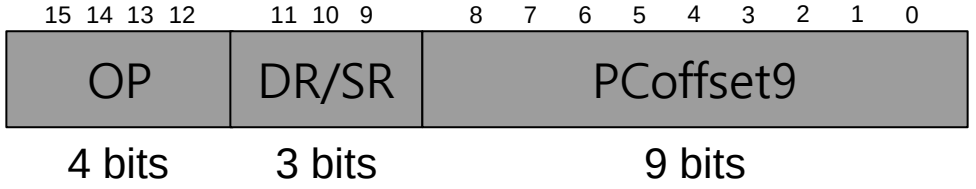


The memory address is only +256 to -255 locations away of the LD or ST instruction

Limitation: The PC-relative addressing mode cannot address far away from the instruction

Indirect Addressing Mode

■LDI (Load Indirect) and STI (Store Indirect)



- ❑OP = opcode
  - ■ E.g., LDI = 1010
  - ■ E.g., STI = 1011
- ❑DR = destination register in LDI
- ❑SR = source register in STI
- ❑LDI: DR ← Memory[Memory[PC† + sign-extend(PCoffset9)]]
- ❑STI: Memory[Memory[PC† + sign-extend(PCoffset9)]] ← SR

† This is the incremented PC

LDI in LC-3

- LDI assembly and machine code

LC-3 assembly

```
LDI R3, 0x1CC
```

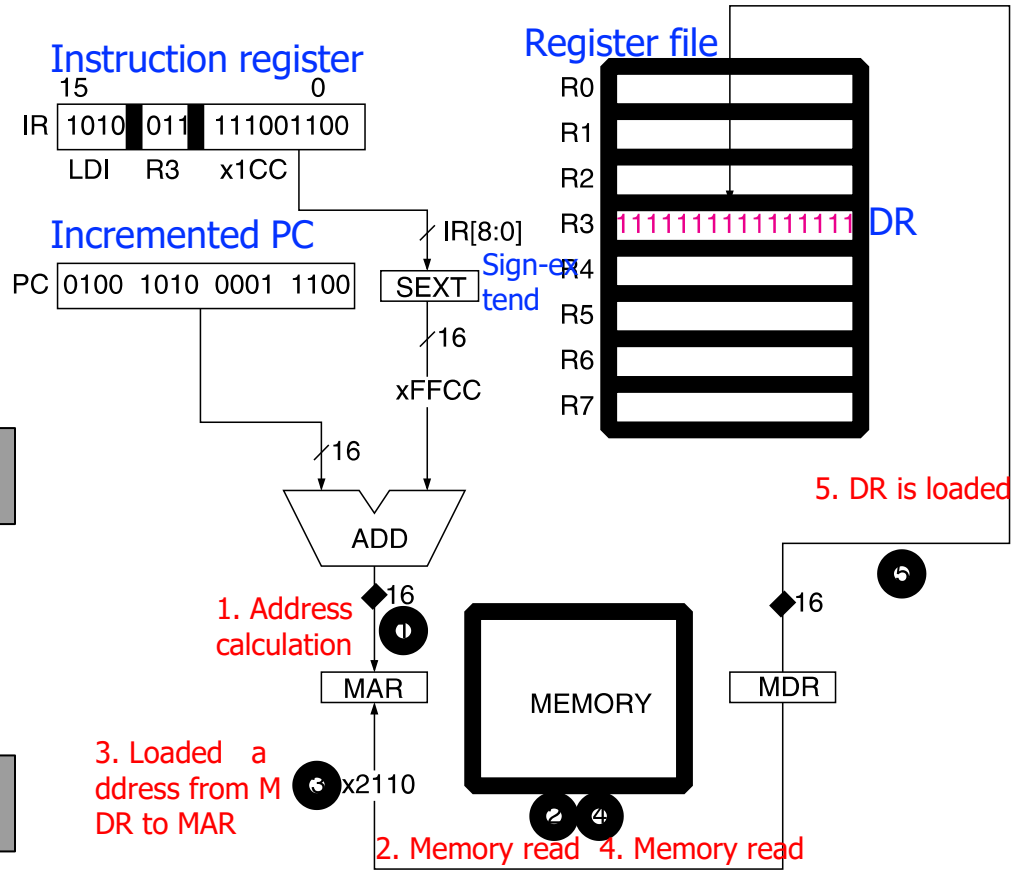
Field Values

OP	DR	PCoffset9
A	3	0x1CC

Machine Code

OP	DR	PCoffset9
1 0 1 0	0 1 1	1 1 1 0 0 1 1 0 0

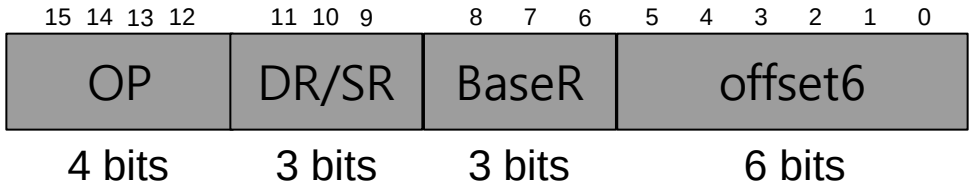
15 12 11 9 8 0



Now the address of the operand can be anywhere in the memory

Base+Offset Addressing Mode

■LDR (Load Register) and STR (Store Register)



❑OP = opcode
- E.g., LDR = 0110
- E.g., STR = 0111

❑DR = destination register in LDR
❑SR = source register in STR

❑LDR: DR ← Memory[BaseR + sign-extend(offset6)]

❑STR: Memory[BaseR + sign-extend(offset6)] ← SR

LDR in LC-3

- LDR assembly and machine code

LC-3 assembly

```
LDR R1, R2, 0x1D
```

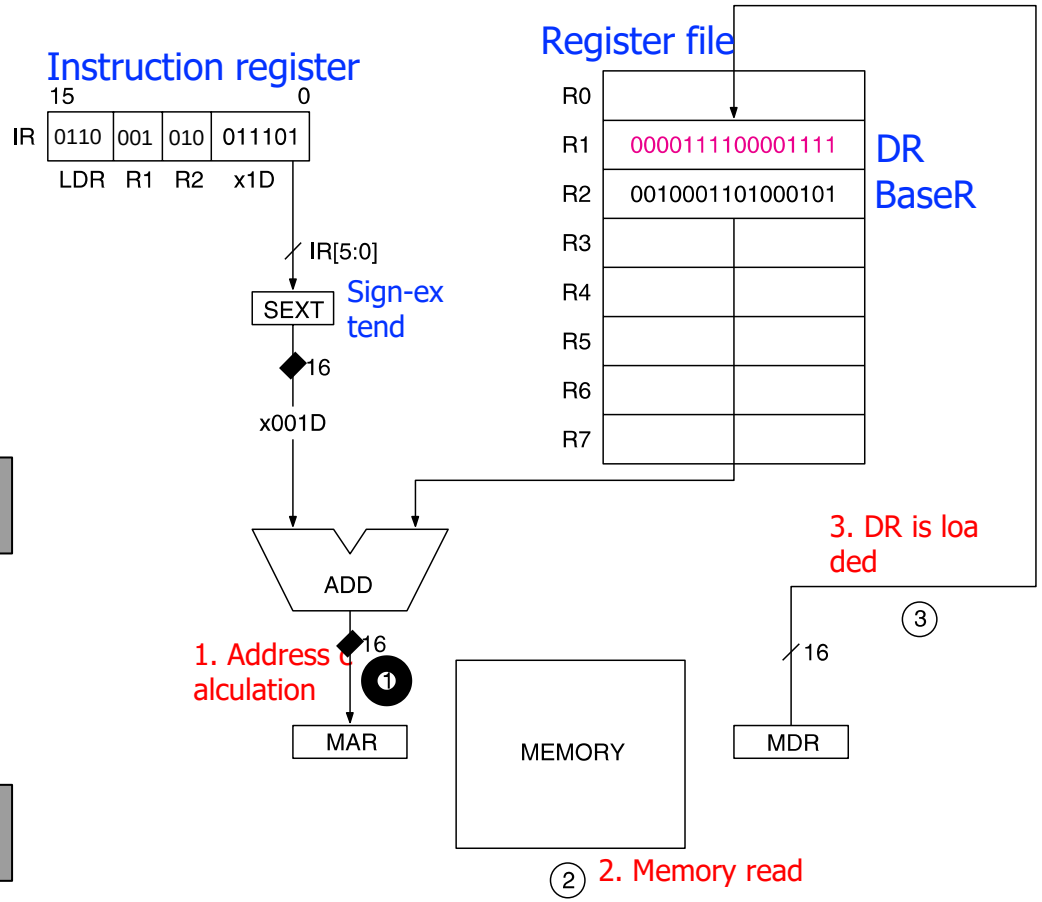
Field Values

OP	DR	BaseR	offset6
6	1	2	0x1D

Machine Code

OP	DR	BaseR	offset6
0 1 1 0	0 0 1	0 1 0	0 1 1 1 0 1

15 12 11 9 8 6 5 0



Again, the address of the operand can be anywhere in the memory

An Example Program in LC-3

High-level code

```
a    = A[0];
c    = a + b - 5;
B[0] = c;
```

LC-3 registers

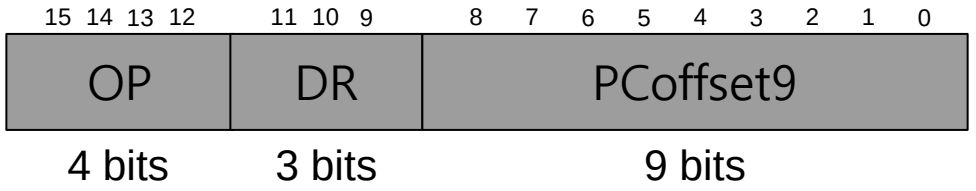
```
A = R0
b = R2
B = R1
```

LC-3 assembly

```
LDR  R5, R0, #0
ADD  R6, R5, R2
ADD  R7, R6, #-5
STR  R7, R1, #0
```

Immediate Addressing Mode

- LEA (Load Effective Address)



  - OP = 1110
  - DR = destination register
  - LEA: DR ← PC† + sign-extend(PCoffset9)

What is the difference from PC-Relative addressing mode?

Answer: Instructions with PC-Relative mode access memory, but LEA does not → Hence the name *Load Effective Address*

† This is the incremented PC

LEA in LC-3

- LEA assembly and machine code

LC-3 assembly

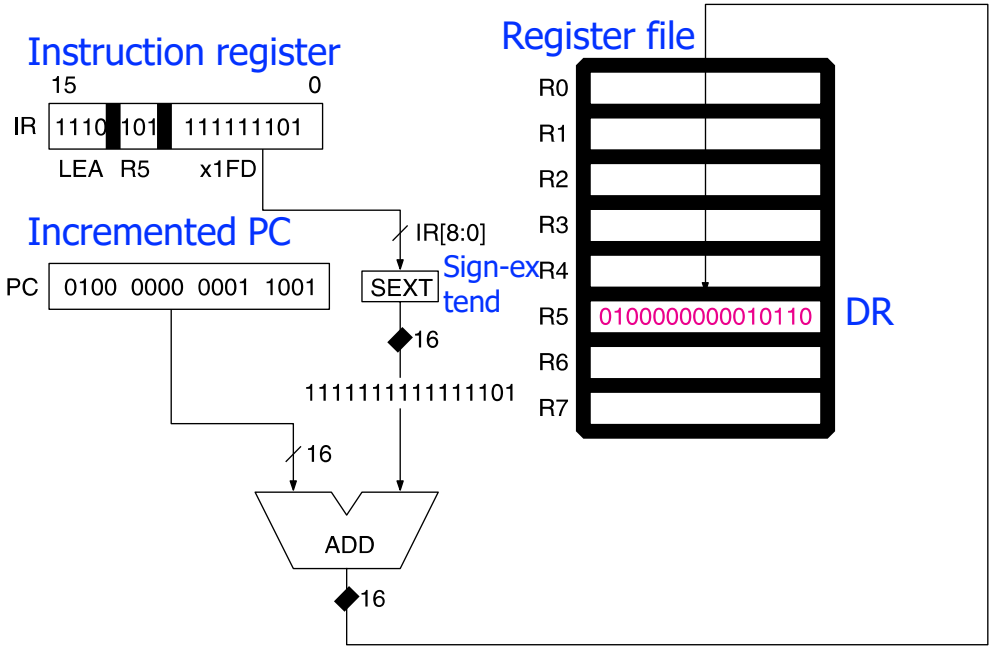
```
LEA R5, #-3
```

Field Values

OP	DR	PCoffset9
E	5	0x1FD

Machine Code

OP	DR	PCoffset9
1 1 1 0	1 0 1	1 1 1 1 1 1 1 0 1
15 12	11 9	8 0



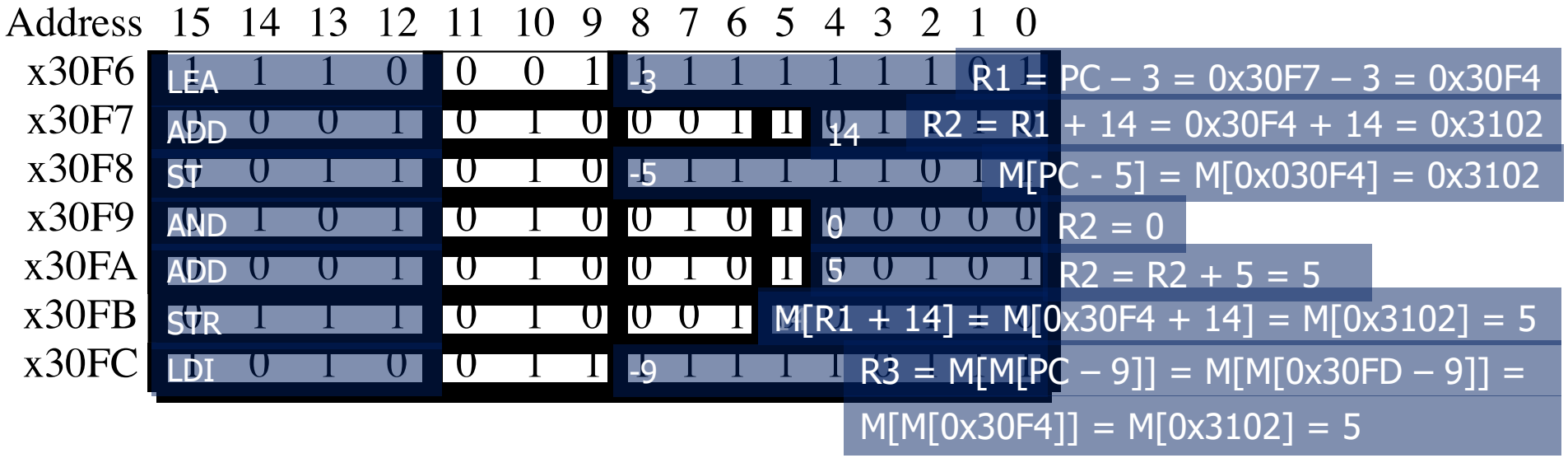
Addressing Example in LC-3

- What is the final value of R3?

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x30F6	1	1	1	0	0	0	1	1	1	1	1	1	1	1	0	1	`R1<- PC-3`
x30F7	0	0	0	1	0	1	0	0	0	1	1	0	1	1	1	0	`R2<- R1+14`
x30F8	0	0	1	1	0	1	0	1	1	1	1	1	1	0	1	1	`M[x30F4]<- R2`
x30F9	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	`R2<- 0`
x30FA	0	0	0	1	0	1	0	0	1	0	1	0	0	1	0	1	`R2<- R2+5`
x30FB	0	1	1	1	0	1	0	0	0	1	0	0	1	1	1	0	`M[R1+14]<- R2`
x30FC	1	0	1	0	0	1	1	1	1	1	1	1	0	1	1	1	`R3<- M[M[x30F4 ]]`

Addressing Example in LC-3

- What is the final value of R3?



- The final value of R3 is 5